

Question description

A company has n employees, who form a tree hierarchy where each employee has a boss, except for the general director. Your task is to process q queries of the form: who is employee x 's boss k levels higher up in the hierarchy?

Input

The first input line has two integers n and q : the number of employees and queries. The employees are numbered $1, 2, \dots, n$, and employee 1 is the general director. The next line has $n-1$ integers $e_2, e_3, \dots, e_n$: for each employee $2, 3, \dots, n$ their boss. Finally, there are q lines describing the queries. Each line has two integers x and k : who is employee x 's boss k levels higher up?

Output

Print the answer for each query. If such a boss does not exist, print -1 .

Constraints

- $1 \leq n, q \leq 2 \cdot 10^5$
- $1 \leq e_i \leq i-1$
- $1 \leq x \leq n$
- $1 \leq k \leq n$

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 3
1 3 2 2
4 2
1 2
1 3
```

EXPECTED OUTPUT

```
1
-1
-1
```

Test Case 2

INPUT (STDIN)

```
5 3
1 1 3 3
4 1
4 2
4 3
```

EXPECTED OUTPUT

```
3
1
-1
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void link(int i,int j)
```

▼ Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
2	420	50

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAXN 200005
5 #define LOG 18
6
7 int up[MAXN][LOG];
8 int depth[MAXN];
9
10 int main() {
11     int n, q;
12     if (scanf("%d %d", &n, &q) != 2) return 0;
13
14     for (int i = 2; i <= n; i++) {
15         int p;
16         scanf("%d", &p);
17         up[i][0] = p;
18         depth[i] = depth[p] + 1;
19         for (int j = 1; j < LOG; j++) {
20             up[i][j] = up[up[i][j-1]][j-1];
21         }
22     }
23
24     for (int i = 0; i < q; i++) {
25         int x, k;
```

Ln 39, Col 1 Spaces: 4 UTF-8 CRLF C

```
26     scanf("%d %d", &x, &k);
27     if (k > depth[x]) {
28         printf("-1\n");
29     } else {
30         for (int j = LOG - 1; j >= 0; j--) {
31             if (k & (1 << j)) {
32                 x = up[x][j];
```

```
33         }
34     }
35     printf("%d\n", x);
36 }
37 }
38 return 0;
39 }
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5 3

1 3 2 2

4 2

1 2

1 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree16.exe

1

-1

-1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

5 3

1 1 3 3

4 1

4 2

4 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree16.exe

3

1

-1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>